

**TRABAJO PRÁCTICO INTEGRADOR:
ESTRUCTURAS DE DATOS, CONTRATO
EQUALS/HASHCODE Y COMPLEJIDAD ALGORÍTMICA**

Asignatura:

Desarrollo de Software

Comisión: 3K10

Integrantes:

- Geronimo Rubert
- Kevin Diaz
- Leandro Tarditi
- Alvaro Illanes
- Lucia Vazquez
- Agustina Moreno

Trabajo Práctico Integrador: Estructuras de Datos, Contrato Equals/HashCode y Complejidad

Algorítmica Asignatura: Desarrollo de Software

Modalidad: en grupo

Criterio de Evaluación y Política de IA: Se permite el uso de herramientas de Inteligencia Artificial (LLMs). Sin embargo, el práctico está diseñado para evaluar el **razonamiento crítico, el análisis de estados de memoria y la justificación técnica**. Las respuestas genéricas generadas por IA sin prueba de trazabilidad manual o sin responder a los escenarios de borde específicos serán penalizadas.

Criterios Generales de Entrega

- Entregar un informe en formato PDF publicado en un repositorio de GitHub.
- Incluir fragmentos de código fuente en Java y los diagramas/trazas explicativas

solicitadas.

Consigna 1: Diagnóstico de Bug Silencioso y Análisis de Memoria

Un equipo de desarrollo junior implementó el siguiente sistema para gestionar el acceso de empleados a un edificio mediante tarjetas de identificación mutable. El código no lanza excepciones en tiempo de ejecución, pero presenta comportamientos anómalos en producción.

```
import java.util.*;

class TarjetaAcceso {
    private String codigo;
    private int nivelAcceso;

    public TarjetaAcceso(String codigo, int nivelAcceso) {
        this.codigo = codigo;
        this.nivelAcceso = nivelAcceso;
    }

    public void setNivelAcceso(int nivelAcceso) {
        this.nivelAcceso = nivelAcceso;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        TarjetaAcceso tarjeta = (TarjetaAcceso) o;
        return nivelAcceso == tarjeta.nivelAcceso && Objects.equals(codigo,
tarjeta.codigo);
    }

    @Override
    public int hashCode() {
        return Objects.hash(codigo, nivelAcceso);
    }
}

public class ControlAcceso {
```

```

public static void main(String[] args) {
    Set<TarjetaAcceso> tarjetasValidas = new HashSet<>();[cite: 3]
    TarjetaAcceso t1 = new TarjetaAcceso("ACC-101", 2);

    tarjetasValidas.add(t1);

    // Modificación posterior a la inserción
    t1.setNivelAcceso(5);

    System.out.println("¿Tarjeta válida en sistema?: " +
tarjetasValidas.contains(t1));
    System.out.println("Cantidad de tarjetas registradas: " +
tarjetasValidas.size());
}
}

```

Responder y Justificar:

1. **Traza del Estado Interno:** Explique paso a paso qué ocurre dentro de la estructura HashSet:
 - Al ejecutar tarjetasValidas.add(t1).
 - Al ejecutar t1.setNivelAcceso(5).
 - Al ejecutar tarjetasValidas.contains(t1).

tarjetasValidas.add(t1)

El HashSet por dentro es un HashMap donde la tarjeta hace de clave. Le pide su hashCode(), con ese número elige un casillero de su tabla y ahí arma un nodo que guarda dos cosas: la tarjeta y el hash recién calculado. Ese hash queda congelado: **no se recalcula nunca más**.

-1387020803

Se posiciona en bucket # -1387020803.

t1.setNivelAcceso(5)

-1387020800

Como nivelAcceso entra en el cálculo del hashCode() (# -13877020800), la tarjeta pasa a devolver otro número, que apunta a otro casillero. Pero el HashSet no lo registra al objeto (# -1387020803). La tarjeta se queda donde estaba, en el anterior. Se rompió el contrato de hashCode() (mientras el objeto viva adentro del Set, tiene que devolver siempre lo mismo) y la estructura quedó sin sentido, sin que salte ningún error. Por eso el bug es silencioso: no se rompe nada, empieza a mentir.

tarjetasValidas.contains(t1)

-1387020800

Al invocar `contains(t1)`, el `HashSet` recalcula el hash del objeto y, a partir del valor mutado, determina un bucket distinto de aquel donde el elemento fue almacenado. Como ese bucket está vacío, la búsqueda finaliza allí y devuelve `false`, sin llegar a invocar `equals()`. Aunque hubiera caído en el mismo casillero, el `HashMap` compara primero el hash guardado contra el nuevo, y como no coinciden la descarta igual.

2. Explicación del Cajón (Bucket): ¿En qué número/ubicación de cajón quedó guardado el objeto y en cuál lo busca Java al hacer `contains()`?

En la memoria interna del `HashSet`, el objeto sigue guardado en el bucket original -1387020803; mientras que cuando se produce la mutación del mismo, y se pide verificar el método `contains()`, el sistema calcula el nuevo hash -1387020800 y busca el objeto en el mismo.

3. Fuga de Memoria (Memory Leak): ¿Por qué este patrón genera una fuga de memoria si se ejecuta repetidamente en un servidor que opera 24/7?

Este patrón puede generar una fuga de memoria porque, al cambiar el valor de `nivelAcceso`, también cambia el `hashCode()` de `t1`. Sin embargo, el objeto permanece en el bucket original del `HashSet`. Por eso las operaciones como `.remove(t1)` pueden no encontrarlo, sería como buscarlo en el cajón equivocado. Como el `HashSet` mantiene una referencia activa a él, el recolector de basura no puede liberarlo de la memoria del Heap. Si esto se repite continuamente, pueden acumularse objetos inaccesibles dentro del `HashSet`, aumentando el consumo de memoria y pudiendo provocar el colapso de la aplicación.

4. Refactorización Obligatoria: Aplique dos soluciones arquitectónicas distintas para corregir el problema (una basada en inmutabilidad y otra ajustando el criterio de la entidad).

Solucion basada en inmutabilidad:

Convertimos la clase en un Java Record para asegurar la inmutabilidad de sus componentes por diseño.

```
public record TarjetaAcceso(String codigo, int nivelAcceso) {}
```

Al no poder modificar `codigo` ni `nivelAcceso` después de crear la tarjeta, tampoco puede cambiar el `hashCode()` mientras el objeto se encuentra dentro del `HashSet`.

Solucion basada en ajustar el criterio de la entidad:

Modificamos el criterio de identidad de `TarjetaAcceso` para que dependa únicamente de su identificador único, `codigo`. De esta manera, `nivelAcceso` puede modificarse sin afectar el `hashCode()` ni la ubicación del objeto dentro del `HashSet`.

```
© Main.java  © TarjetaAcceso.java x
1  import java.util.Objects;
2
3  class TarjetaAcceso {
4      private String codigo;
5      private int nivelAcceso;
6
7      public TarjetaAcceso(String codigo, int nivelAcceso) {
8          this.codigo = codigo;
9          this.nivelAcceso = nivelAcceso;
10     }
11
12     public void setNivelAcceso(int nivelAcceso) {
13         this.nivelAcceso = nivelAcceso;
14     }
15
16     @Override
17     public boolean equals(Object o) {
18         if (this == o) return true;
19         if (o == null || getClass() != o.getClass()) return false;
20
21         TarjetaAcceso tarjeta = (TarjetaAcceso) o;
22
23         return Objects.equals(codigo, tarjeta.codigo);
24     }
25
26     @Override
27     public int hashCode() {
28         return Objects.hash(codigo);
29     }
30 }
31 //Solucion basada en ajustar el criterio de la entidad
```

Consigna 2: Selección y Diseño de Arquitectura de Colecciones

Diseñe el backend de un módulo para una plataforma de *E-Commerce*. Debe elegir la interfaz y la implementación concreta del Java Collections Framework (java.util) más eficiente para cada una de las siguientes tres funcionalidades:

Requerimiento 1: Sistema de Auditoría de Transacciones Registrar operaciones en el orden exacto en que ocurren. Se requiere insertar miles de registros por segundo al final de la colección y recorrerlos secuencialmente.
Requerimiento 2: Catálogo de Ofertas Relámpago Mantener los productos ordenados automáticamente por precio de menor a mayor. Debe permitir consultar eficientemente el producto más barato y el más caro.
Requerimiento 3: Búsqueda de Sesiones de Usuarios Activos Buscar instantáneamente la sesión de un usuario dado su Token (String único) entre 500,000 usuarios conectados en simultáneo.

Completar la siguiente tabla comparativa y justificar la decisión con la complejidad algorítmica Big O:

Requerimiento	Interfaz Seleccionada (List , Set , Map , Queue) PDF	Implementación Concreta (ArrayList , TreeSet , HashMap , etc.) PDF	Complejidad Búsqueda	Complejidad Inserción	Justificación Técnica de la Elección
1. Auditoría					
2. Ofertas					
3. Sesiones					

Requerimiento	Interfaz Seleccionada	Implementación Concreta	Complejidad Búsqueda	Complejidad Inserción
1.Auditoría	List	ArrayList	$O(n)$	$O(1)$
2.Ofertas	Set	TreeSet	$O(\log n)$	$O(\log n)$
3.Sesiones	Map	Hashmap	$O(1)$	$O(1)$

Justificación Técnica de la Elección:

1. Sistema de Auditoría de Transacciones:

Se necesita una estructura secuencial que conserve el orden de llegada de las operaciones de pago. Por este motivo utilizamos List, para que se registren en el orden exacto en que ocurren, permitiendo que estén duplicados. En el caso de SET: no permite duplicados, por ende si hay transacciones idénticas (un Usuario realiza dos compras seguidas del mismo producto), si utilizamos el SET, la segunda transacción se descartaría.

Luego, como el requerimiento exige que se registren de manera continua al final de la colección, ArrayList resuelve en tiempo constante.

2. Catálogo de Ofertas Relámpago:

Utilizamos SET ya que un catálogo de ofertas no debe tener productos duplicados. Si utilizamos un LIST, tendríamos que verificar manualmente que no estén los productos múltiples veces. Aparte, el requerimiento pide mantener los productos ordenados por precio. Al implementar SET con TREESET, los productos se mantienen ordenados, y evita tener que ordenarlos manualmente cada vez que se agrega o modifica una oferta.

3. Búsqueda de Sesiones de Usuarios Activos:

El requerimiento pide buscar la sesión de un usuario a partir de su Token entre 500000 usuarios, por ende, utilizamos MAP ya que esta está diseñada para almacenar datos asociados a una "Clave" única a un "Valor". Si utilizáramos una List o Set para guardar este número de usuarios conectados, cada vez que un usuario realice una petición, el servidor tendría que recorrer toda la colección comparando uno por uno los Tokens. Luego, al utilizar HASHMAP, este método calcula el índice exacto donde se guardó la sesión, y gracias a esto, se realiza en un tiempo promedio constante, lo cual permite que el sistema responda de forma instantánea.

Consigna 3: Optimización Algorítmica ($O(n^2)$ -> $O(n)$)

Un desarrollador escribió el siguiente método para encontrar el primer carácter que no se repite dentro de una cadena de texto (ejemplo: en "barco", el primero es 'b'; en "abarca", el primero es 'r').

```
public static Character primerCaracterNoRepetidoLento(String texto) {  
    // Algoritmo ineficiente  $O(n^2)$   
    for (int i = 0; i < texto.length(); i++) {  
        char c = texto.charAt(i);  
        boolean duplicado = false;  
        for (int j = 0; j < texto.length(); j++) {  
            if (i != j && c == texto.charAt(j)) {  
                duplicado = true;  
                break;  
            }  
        }  
        if (!duplicado) return c;  
    }  
    return null;  
}
```

Responder y Justificar:

1. **Refactorización a $O(n)$:** Escriba una versión optimizada del método utilizando la colección adecuada de java.util (HashMap, LinkedHashMap, o HashSet) que resuelva el problema realizando como máximo dos recorridos lineales ($O(n)$).

Esta consigna busca que detectemos por qué el código original es $O(n^2)$ (ya que tiene un for dentro de un for) y lo reemplacemos por alguna solución que tenga una colección para evitar recorrer toda la cadena por cada carácter.

La opción más clara sería `HashMap<Character, Integer>`, porque nos dice cuántas veces aparece cada carácter.

```
1  import java.util.HashMap;
2  import java.util.Map;
3
4  public class Main {
5
6      public static Character primerCaracterNoRepetidoLento(String texto) {
7          for (int i = 0; i < texto.length(); i++) {
8              char c = texto.charAt(i);
9              boolean duplicado = false;
10
11              for (int j = 0; j < texto.length(); j++) {
12                  if (i != j && c == texto.charAt(j)) {
13                      duplicado = true;
14                      break;
15                  }
16              }
17
18              if (!duplicado) {
19                  return c;
20              }
21          }
22
23          return null;
24      }
25
26      public static Character primerCaracterNoRepetido(String texto) {
27
28          // primer recorrido, contamos cuántas veces aparece cada carácter
29          Map<Character, Integer> frecuencias = new HashMap<>();
30
31          for (int i = 0; i < texto.length(); i++) {
32              char c = texto.charAt(i);
33              frecuencias.put(c, frecuencias.getOrDefault(c, 0) + 1);
34          }
```

```

35
36     // segundo recorrido, buscamos el primer carácter que su frecuencia sea 1
37     for (int i = 0; i < texto.length(); i++) {
38         char c = texto.charAt(i);
39
40         if (frecuencias.get(c) == 1) {
41             return c;
42         }
43     }
44
45     return null;
46 }
47
48 Run | Debug
49 public static void main(String[] args) {
50     System.out.println(primerCaracterNoRepetido(texto: "barco")); // b
51     System.out.println(primerCaracterNoRepetido(texto: "abarca")); // r
52 }
53 }

```

2. **Análisis de Colisiones:** Si se utiliza un HashMap internamente, explique qué ocurre si dos caracteres distintos generan el mismo hashCode() (Colisión de Hash) y cómo la estructura resuelve este problema sin perder información.

Si dos caracteres distintos generan el mismo hashCode(), ocurre una colisión de hash, esto no quiere decir que las claves sean iguales, sino que las dos van a parar inicialmente al mismo bucket del HashMap. El HashMap puede guardar varias entradas dentro del mismo bucket. Para saber cuál es la clave correcta no se fija solamente en el hashCode(), sino que también usa equals() para comparar las claves. De esta forma dos caracteres diferentes como 'a' y 'b' pueden estar en el mismo bucket sin que se pierda información. Además, si hay muchas colisiones Java puede convertir ese bucket en un árbol para que la búsqueda sea más rápida.

Consigna 4: Caso Teórico de Borde y Colisiones

Considere las siguientes dos instancias de String:

- String s1 = "FB";
- String s2 = "Ea";

Sabiendo que s1.hashCode() == s2.hashCode() devuelve true (ambos dan el entero 2236):

1. Si guardamos ambas cadenas en un `HashSet<String>`, ¿cuántos elementos contendrá la colección? La colección `HashSet` contendrá 2 elementos.

Explique cómo utiliza Java los métodos `hashCode()` y `equals()` internamente para determinar si s2 debe ingresarse o desecharse.

Primero cuando ingresa s1, java llama a `"FB".hashCode()`. Esto da 2236 y va al cajón 2236, cómo está vacío guarda a "FB" ahí y el tamaño del Set queda en 1. Luego ingresa s2, java llama a `"Ea".hashCode()`. Esto da 2236 también. Java va al cajón 2236 y observa que ya hay algo guardado, entonces se produce una colisión de Hash. Para resolver esta colisión y decidir si lo guarda o lo descarta, Java invoca internamente el método `equals()` comparando el elemento existente con el nuevo: `"FB".equals("Ea")`. Como el contenido de las cadenas es distinto, el método devuelve false. Al confirmar matemáticamente que no son el mismo objeto, lo que sucede es que Java no desecha a "Ea". Lo mete en el mismo cajón 2236, armando como una especie de lista que sería un nodo al lado del otro junto a "FB". Por lo tanto ambos están juntos y el tamaño final es 2.

2. ¿Cómo afecta la acumulación de colisiones de hash al rendimiento teórico de un `HashMap` en el peor escenario posible (de $O(1)$ a $O(n)$ o $O(\log n)$)?

Casi siempre un `HashMap` funciona en $O(1)$ pero el problema o el peor escenario aparece cuando muchas claves distintas caen en el mismo bucket o sea muchas colisiones, cuando pasa eso el bucket deja de comportarse como un acceso directo y pasa a ser como una estructura que hay que recorrer. Ahí se procede a usar el método **equals** hasta encontrar la clave que es correcta. Si todas las claves terminan colisionando en un mismo bucket, este terminaría comportándose como una lista enlazada de n elementos y todas las operaciones de búsqueda, inserción y de borrado pasarían a ser $O(n)$ y técnicamente el `HashMap` pierde lo que lo hace útil.

Desde Java 8 en adelante esto se disminuye porque cuando un bucket acumula más de 8 elementos y la tabla ya tiene al menos 64 buckets o más, java concluye que hay claves que están de más y agrandar la tabla no va a resolver el problema entonces lo convierte al bucket en un árbol rojo-negro ordenado por `hashCode`. Por eso el peor caso deja de ser $O(n)$ y pasa a ser $O(\log n)$